

Assignment 5: Quicksort Algorithm: Implementation, Analysis, and Randomization

Fathiya Adan

University of Cumberlands

Algorithms and Data Structures (MSCS-532-B01)

Michael Solomon

March 29, 2026

Introduction

Quicksort is one of the most efficient and widely used sorting algorithms in computer science due to its strong average-case performance and in-place operation. It follows a divide-and-conquer approach in which an array is partitioned around a selected pivot element, and the resulting subarrays are recursively sorted. Despite its efficiency, the performance of Quicksort is highly dependent on the pivot selection strategy. Poor pivot choices can lead to unbalanced partitions, significantly degrading performance.

The purpose of this assignment is to implement both deterministic and randomized versions of Quicksort, analyze their theoretical time complexity, and empirically evaluate their performance across different input distributions. The deterministic version selects a fixed pivot, while the randomized version selects the pivot randomly to improve performance stability. This report presents the implementation details, theoretical analysis, and experimental findings, demonstrating the impact of pivot selection on algorithm performance.

Algorithm Background

Quicksort operates by selecting a pivot element and partitioning the array such that elements smaller than or equal to the pivot are placed on one side, and larger elements are placed on the other. After partitioning, the pivot is in its correct sorted position, and the algorithm recursively applies the same process to the left and right subarrays.

The effectiveness of Quicksort depends on how balanced the partitions are. Balanced partitions result in shallow recursion and efficient execution, while unbalanced partitions lead to deep recursion and poor performance. Deterministic Quicksort uses a fixed pivot rule, which can be problematic for certain input patterns. Randomized Quicksort improves upon this by selecting pivots randomly, reducing the likelihood of consistently poor partitioning.

Implementation Details

Two versions of Quicksort were implemented in Python: deterministic Quicksort and randomized Quicksort. Both versions share the same partition procedure, but they differ in the way the pivot is selected.

Deterministic Quicksort

The deterministic implementation uses the last element of the current subarray as the pivot. The partition function scans the subarray from left to right, maintaining an index that separates smaller elements from larger elements. Whenever an element less than or equal to the pivot is found, it is swapped into the lower partition. After the scan is complete, the pivot is swapped into its correct position. The function then returns the pivot index, which is used by the recursive Quicksort procedure to sort the subarrays on the left and right.

This implementation is straightforward and efficient on many inputs. However, because the pivot selection rule is fixed, the algorithm is vulnerable to inputs that are already sorted or nearly sorted. In such cases, the pivot often ends up at one extreme of the array, resulting in highly unbalanced recursive calls.

Randomized Quicksort

The randomized version modifies only the pivot selection step. Before partitioning, a random index between the low and high bounds of the current subarray is selected. The element at that random index is swapped with the last element, and then the regular partition function is applied. This means that the rest of the algorithm remains identical to the deterministic version.

The advantage of this design is that it preserves the simplicity of the original partition logic while greatly improving the expected balance of partitions. Because the pivot is chosen randomly, the algorithm is much less likely to encounter the same poor partition pattern repeatedly. As a result, randomized Quicksort is more stable across different input distributions.

Supporting Functions

To perform empirical analysis, helper functions were created to generate arrays with different characteristics:

- random arrays filled with randomly generated integers,
- sorted arrays in ascending order,
- reverse-sorted arrays in descending order.

An additional timing function was implemented to measure the running time of each sorting procedure. To ensure fairness, each algorithm was tested on a copy of the original input array rather than modifying the same array in place across multiple runs.

Python Source Code

The following code was used to implement both algorithms and perform the empirical analysis:

```
import random
import time

def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1

    for j in range(low, high):
        if arr[j] <= pivot:
            i += 1
```

```
arr[i], arr[j] = arr[j], arr[i]
```

```
arr[i + 1], arr[high] = arr[high], arr[i + 1]  
return i + 1
```

```
def quicksort(arr, low, high):  
    if low < high:  
        pi = partition(arr, low, high)  
        quicksort(arr, low, pi - 1)  
        quicksort(arr, pi + 1, high)
```

```
def randomized_partition(arr, low, high):  
    random_index = random.randint(low, high)  
    arr[random_index], arr[high] = arr[high], arr[random_index]  
    return partition(arr, low, high)
```

```
def randomized_quicksort(arr, low, high):  
    if low < high:  
        pi = randomized_partition(arr, low, high)  
        randomized_quicksort(arr, low, pi - 1)  
        randomized_quicksort(arr, pi + 1, high)
```

```
def generate_random_array(size):  
    return [random.randint(1, 10000) for _ in range(size)]
```

```
def generate_sorted_array(size):  
    return list(range(size))
```

```
def generate_reverse_sorted_array(size):  
    return list(range(size, 0, -1))
```

```
def test_quicksort_performance(sort_function, arr):  
    arr_copy = arr.copy()
```

```

start_time = time.time()
sort_function(arr_copy, 0, len(arr_copy) - 1)
end_time = time.time()
return end_time - start_time

if __name__ == "__main__":
    sizes = [100, 300, 500, 700, 900]

    for size in sizes:
        print(f"\nArray size: {size}")

        random_arr = generate_random_array(size)
        sorted_arr = generate_sorted_array(size)
        reverse_arr = generate_reverse_sorted_array(size)

        det_random = test_quicksort_performance(quicksort, random_arr)
        rand_random = test_quicksort_performance(randomized_quicksort, random_arr)

        det_sorted = test_quicksort_performance(quicksort, sorted_arr)
        rand_sorted = test_quicksort_performance(randomized_quicksort, sorted_arr)

        det_reverse = test_quicksort_performance(quicksort, reverse_arr)
        rand_reverse = test_quicksort_performance(randomized_quicksort, reverse_arr)

        print(f"Random input - Deterministic: {det_random:.6f}s, Randomized:
{rand_random:.6f}s")
        print(f"Sorted input - Deterministic: {det_sorted:.6f}s, Randomized: {rand_sorted:.6f}s")
        print(f"Reverse input - Deterministic: {det_reverse:.6f}s, Randomized:
{rand_reverse:.6f}s")

```

Time Complexity Analysis

Best-Case Time Complexity

The best case for Quicksort occurs when the pivot divides the array into two nearly equal halves at every recursive step. In this case, the recursion tree has a depth of approximately $\log n$, because the problem size is halved at each level. At each level of recursion, the partition operation examines all n elements across the subproblems combined. Therefore, the total running time is proportional to $n \log n$.

This can be represented by the recurrence relation:

$$T(n) = 2T(n/2) + O(n)$$

Solving this recurrence gives:

$$T(n) = O(n \log n)$$

This is the ideal behavior of Quicksort and explains why the algorithm is considered highly efficient in practice when good pivot choices are made.

Average-Case Time Complexity

The average case also has time complexity $O(n \log n)$. Even though the partitions are not perfectly balanced at every recursive call, they tend to be reasonably balanced over many recursive steps, especially when the pivot is selected randomly or when the input order is not adversarial. On average, the recursion depth remains logarithmic, and each level still performs a total of $O(n)$ work through partitioning.

The average-case efficiency of Quicksort is one of the main reasons for its popularity. Compared with algorithms such as Merge Sort, Quicksort often performs better in practice because of its cache efficiency and in-place partitioning, despite both having $O(n \log n)$ average performance.

Worst-Case Time Complexity

The worst case occurs when the pivot repeatedly produces highly unbalanced partitions. For example, if the pivot is always the smallest or largest element, the partition step produces one subarray of size $n - 1$ and another of size 0. In that case, the recurrence becomes:

$$T(n) = T(n - 1) + O(n)$$

Solving this recurrence gives:

$$T(n) = O(n^2)$$

This worst-case behavior is especially likely in deterministic Quicksort when the pivot is always chosen as the last element and the input array is already sorted or reverse-sorted. Under these conditions, the algorithm effectively degenerates into a nested sequence of scans, causing much slower performance and deeper recursion.

Why Randomization Helps

Randomized Quicksort reduces the chance of worst-case behavior by choosing the pivot randomly. Although the worst-case time complexity remains $O(n^2)$ in theory, the probability of repeatedly selecting extremely poor pivots becomes very small. As a result, the expected running time remains $O(n \log n)$, and the algorithm performs much more consistently across different input patterns.

Randomization does not guarantee that a bad partition will never occur, but it makes such patterns far less likely. This is why randomized Quicksort is often preferred when input order is unpredictable or when protection against adversarial inputs is important.

Space Complexity Analysis

Quicksort is generally considered an in-place sorting algorithm because it does not require an additional array proportional to the input size. The partitioning process rearranges elements directly within the original array, so the main extra memory cost comes from recursion.

In the best and average cases, the recursion depth is $O(\log n)$, so the auxiliary space required by the call stack is $O(\log n)$. In the worst case, however, the recursion depth becomes $O(n)$, which increases the auxiliary space requirement to $O(n)$. This is particularly relevant in Python, where deep recursion can cause a recursion depth error if the call stack grows too large.

Randomized Quicksort has the same formal space complexity as deterministic Quicksort, but in practice it is less likely to reach the worst-case recursion depth because its pivot choices are more balanced on average.

Empirical Analysis

The performance of deterministic and randomized Quicksort was tested on three input distributions:

- random input,
- sorted input,
- reverse-sorted input.

The following array sizes were used:
100, 300, 500, 700, and 900 elements.

The measured running times are presented below.

Experimental Results

Array Size	Input Type	Deterministic Quicksort (s)	Randomized Quicksort (s)
100	Random	0.000154	0.000165
100	Sorted	0.000596	0.000120
100	Reverse-Sorted	0.000621	0.000114
300	Random	0.000305	0.000407

Array Size	Input Type	Deterministic Quicksort (s)	Randomized Quicksort (s)
300	Sorted	0.004630	0.000348
300	Reverse-Sorted	0.003095	0.000391
500	Random	0.000587	0.000667
500	Sorted	0.015771	0.000633
500	Reverse-Sorted	0.011658	0.000668
700	Random	0.000933	0.001356
700	Sorted	0.028419	0.001090
700	Reverse-Sorted	0.019697	0.001036
900	Random	0.000993	0.001347
900	Sorted	0.044004	0.001207
900	Reverse-Sorted	0.030493	0.001194

Discussion of Results

The empirical results strongly support the theoretical analysis of both algorithms.

Random Inputs

For random inputs, both deterministic and randomized Quicksort performed very efficiently. Their running times were quite close for all tested sizes, and both scaled well as the array size increased. In several cases, deterministic Quicksort was slightly faster than randomized Quicksort. This is expected because the deterministic version does not incur the overhead of generating a random pivot index at each partition step.

For example, at array size 500, deterministic Quicksort completed in 0.000587 seconds, while randomized Quicksort took 0.000667 seconds. At size 900, deterministic Quicksort took 0.000993 seconds compared with 0.001347 seconds for the randomized version. These results are consistent with the idea that, on average, both versions operate in $O(n \log n)$ time, but randomization introduces a small constant-factor overhead.

Sorted Inputs

The sorted input results reveal the most important weakness of deterministic Quicksort. Because the deterministic implementation always chooses the last element as the pivot, a sorted array causes the pivot to be the largest element in every recursive call. This leads to extremely unbalanced partitions, where one side contains $n - 1$ elements and the other side contains 0 elements. As a result, the recursion depth increases dramatically and the running time approaches $O(n^2)$.

This effect becomes obvious as the array size grows. At size 100, deterministic Quicksort required 0.000596 seconds, which was already noticeably slower than the randomized version at 0.000120 seconds. At size 500, deterministic Quicksort took 0.015771 seconds, while randomized Quicksort took only 0.000633 seconds. At size 900, the deterministic time rose to 0.044004 seconds, compared with only 0.001207 seconds for randomized Quicksort.

These results clearly show that randomized pivot selection protects the algorithm from the input-order sensitivity that affects deterministic Quicksort. Even on sorted input, randomized Quicksort remained stable and efficient.

Reverse-Sorted Inputs

The reverse-sorted input produced a similar pattern. In deterministic Quicksort, choosing the last element as the pivot again results in extremely poor partitioning, since the pivot becomes the smallest element in the subarray. This leads to the same kind of highly unbalanced recursion and near-worst-case performance.

At size 500, deterministic Quicksort required 0.011658 seconds on reverse-sorted input, while randomized Quicksort required only 0.000668 seconds. At size 900, deterministic Quicksort took 0.030493 seconds, compared with 0.001194 seconds for the randomized version. These results confirm that deterministic Quicksort is highly sensitive not only to sorted input but also to reverse-sorted input when a fixed pivot rule is used.

Stability of Randomized Quicksort

One of the most important conclusions from the experiment is that randomized Quicksort maintained relatively stable performance across all three input distributions. While its times increased gradually as the input size grew, the increase was modest and much more consistent than that of deterministic Quicksort. This stability is precisely the practical benefit of randomization. By reducing dependence on input order, randomized Quicksort produces more predictable performance and greatly lowers the risk of worst-case behavior.

Observations on Practical Performance

The results also illustrate an important practical principle in algorithm design: an algorithm with slightly higher constant overhead may still be preferable if it avoids large performance degradation in difficult cases. Randomized Quicksort may be slightly slower on some random

inputs because of the cost of selecting random pivots, but this extra cost is minimal compared with the dramatic slowdown avoided on sorted and reverse-sorted inputs.

In real-world applications, data is not always random, and input distributions may be partially ordered or even adversarial. Therefore, using randomization can significantly improve robustness without sacrificing much speed in normal cases.

Challenges Encountered

One practical challenge encountered during testing was that deterministic Quicksort can approach Python's recursion depth limit when the input size becomes large and the pivot choices are consistently poor. This happened because sorted inputs caused the recursion depth to grow almost linearly with the array size. This issue itself is a useful demonstration of the theoretical worst case, since it shows that poor partitioning does not only affect running time but can also affect the feasibility of execution in languages with recursion limits.

Another challenge was ensuring fair timing comparisons between the two algorithms. This was addressed by copying the input array before each run, so that both deterministic and randomized Quicksort processed the same original data rather than a previously sorted array. Separate generators for random, sorted, and reverse-sorted arrays also made it possible to examine how input distribution influences algorithm behavior.

A further consideration was selecting array sizes that were large enough to reveal meaningful differences but small enough to execute reliably in the testing environment. Using sizes up to 900 elements provided clear evidence of scaling behavior while avoiding repeated recursion failures.

Conclusion

This assignment provided a detailed exploration of deterministic and randomized Quicksort through implementation, theoretical analysis, and empirical testing. The deterministic version, which selects the last element as the pivot, was simple and efficient on random input but exhibited severe performance degradation on sorted and reverse-sorted arrays. This behavior matched the theoretical worst-case complexity of $O(n^2)$, caused by highly unbalanced partitions and deep recursion.

In contrast, randomized Quicksort demonstrated far more stable performance across all tested input types. Although it introduced a small overhead on random input, it greatly reduced the likelihood of worst-case behavior and maintained performance consistent with the expected $O(n \log n)$ complexity. The experimental results clearly showed that randomization is an effective strategy for improving Quicksort's robustness.

Overall, the assignment confirmed both the power and the limitations of Quicksort. The algorithm is extremely efficient when partitions are balanced, but its performance depends strongly on pivot selection. Randomization offers a practical and elegant improvement that preserves Quicksort's strengths while reducing its main weakness. For this reason, randomized Quicksort is generally the more reliable choice when input order cannot be controlled or predicted.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.